



UNIVERSITI  
TEKNOLOGI  
MARA

Kolej

Pengajian Pengkomputeran,  
Informatik dan Matematik

**UNIVERSITI TEKNOLOGI MARA (UiTM) CAWANGAN MELAKA**

**KAMPUS JASIN**

**SEMESTER 3 SESI 2024/2025  
OCTOBER 2024 – MARCH 2025**

**CSC583  
ARTIFICIAL INTELLIGENCE ALGORITHMS**

**PROJECT TITLE:  
FLOWERS RECOGNITION USING CNN**

**PREPARED BY:**

| STUDENT NAME                       | STUDENT ID |
|------------------------------------|------------|
| 1. MUHAMMAD DANIEL BIN MOHD SAHID  | 2024213154 |
| 2. MAISARAH YASMIN BINTI ISHAK     | 2024807316 |
| 3. NUR FARA'AIN ALIYAH BINTI IMRAN | 2024211146 |
| 4. PUTRI ZULAIKHA BINTI RASHIDI    | 2024441648 |

**CLASS:  
M3CS2303C**

**LINK: [FLOWERS RECOGNITION VIDEO](#)**

**PREPARED FOR:  
MISS FADHLINA IZZAH BINTI SAMAN**

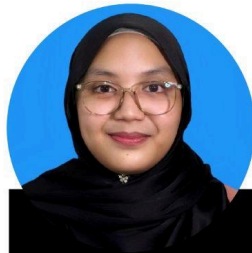
**DUE DATE:  
24<sup>th</sup> JANUARY 2025**

## GROUP MEMBERS



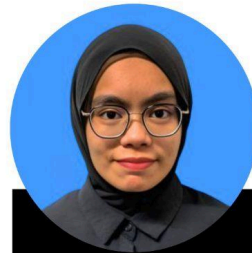
**MUHAMMAD DANIEL  
BIN MOHD SAHID**

**2024213154**



**MAISARAH YASMIN  
BINTI ISHAK**

**2024807316**



**NUR FARA'AIN  
ALIYAH BINTI IMRAN**

**2024211146**



**PUTRI ZULAIKHA  
BINTI RASHIDI**

**2024441648**

## INTRODUCTION

Deep learning is an approach for image recognition and classification issues in the realm of human tasks. It is a widely used technique that has been gradually improved to include additional layers in order to create a strong model for solving the classification problem. One kind of deep learning model created especially for processing and analyzing visual data is called a convolutional neural network (CNN). CNNs, which draw inspiration from the human visual system, are frequently employed in computer vision applications like segmentation, object identification, and image classification.

In this mini project, classification was done on four flower categories namely daisy, tulip, lily, and lilac. Since the prehistoric times, flowers are God's creations that have contributed various benefits for the insects, animals, and humans. This includes feeding the living creatures, as well as being used as ingredients and elements in producing medicines. In the medical industry, flowers are mostly used in herbal medicine. The ancient Egyptians have recorded that their medical properties depended on and based on the use of flowers indicating that flower-based medicine has been a way to produce medicine since then. Even today, modern medicine uses flowers as a healing practice, this includes therapeutics. Among the other benefits of flowers is to be used as a cosmetic and fragrance. With increasing demand for organic and eco-friendly products, flowers are used as a sustainable and chemical-free alternative in beauty and health products. This is all because flowers are a great source of vitamins, antioxidants, and calming qualities that help to nourish the skin, lessen inflammation, and encourage a healthy glow. Therefore, a good understanding of flowers is important in identifying new or rare species when found, so that it can lead to the discovery of innovative uses in fields such as medicine, skin care and perfumery, which ultimately benefit humans.

This project showcases the power of machine learning by delivering highly accurate results through advanced prediction, classification and training techniques. This involves a number of procedures, beginning with an agile approach to data collection and discovery, followed by the network training and testing.

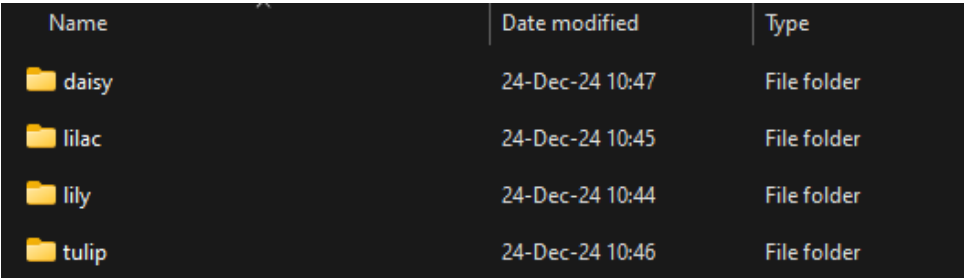
## 1.0 DATASET EXPLANATION

### 1.1 | Objective of The Classification

The objective of this classification is to utilize Convolutional Neural Network (CNNs) for the classification of flowers. The networks will be trained on a diverse set of images, including daisy, lilac, tulip and lily, using MATLAB software. To optimize the accuracy of flower identification, various network settings and structures, such as convolution2dLayer, maxPooling2dLayer, InitialLearnRate, and overall network architecture, will be systematically adjusted. This study aims to determine how these parameter values network designs impact the validation accuracy of CNN.

### 1.2 | Input Data

The input data consists of a comprehensive collection of flower images, where each image represents a specific flower species, which are daisy, lily, tulip and lilac. For each flower category, there are 101 images. The 100 images will be used for the training and validation phases, while one image from each category will be set aside for the testing phase. This results in a total of 404 images across the four categories. All these folders were kept within a main folder named “Flowers,” under which the various types of flowers were categorized.



| Name  | Date modified   | Type        |
|-------|-----------------|-------------|
| daisy | 24-Dec-24 10:47 | File folder |
| lilac | 24-Dec-24 10:45 | File folder |
| lily  | 24-Dec-24 10:44 | File folder |
| tulip | 24-Dec-24 10:46 | File folder |

**Figure 1.2 (a):** Folders for each classification

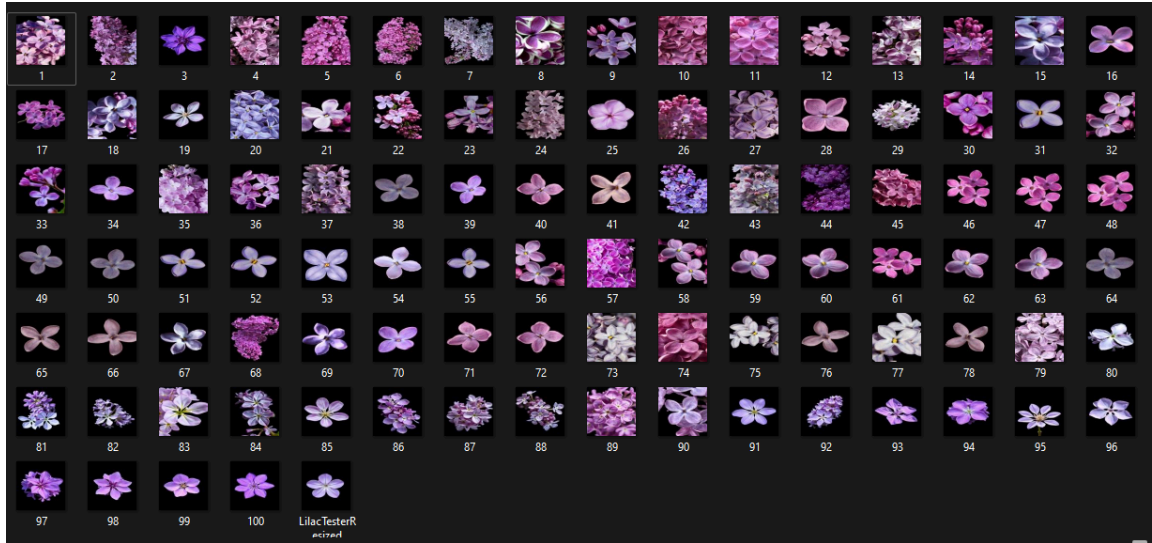
During the initial collection process, these images were collected from many sources available on the internet, and all of them are from the free and copyright free sources, such as Pixabay, Pinterest, Freepik and Unsplash. The collected images vary in resolution, file type, background and angle, providing a rich source of information for

CNN models. To prepare the images for training, each image undergoes a preprocessing step. Each image is resized to 100x100 pixels and converted to jpg file type by using Matlab's function. Additionally, the background of all images is removed and replaced with a black background, represented by the hex code #000000. The background removal is achieved using tools like Canva and free websites such as Remove.bg, which allow for efficient and accurate background replacement. Image resolution is also enhanced using tools such as Canva and Remini, which help improve image quality and clarity. This preprocessing step ensures uniformity across the dataset, helping the CNN model focus solely on the features of the flowers.

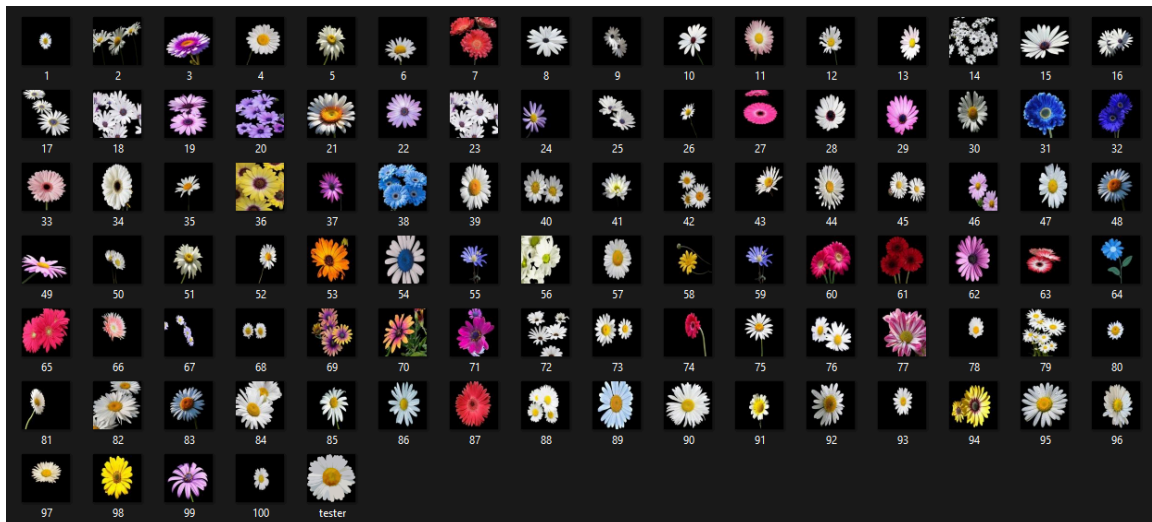
```
% These scripts standardize the .jpg images
Dir = 'D:\Flower Classification\Flowers\daisy'; % Directory containing
images for a specific label
imagefiles = dir(fullfile(Dir, '*.jpg'));
for i = 1 : length(imagefiles)
    filename = strcat(Dir, '\', imagefiles(i).name);
    sf = imread(filename);
    sf = imresize(sf, [100 100]); % Standard size of the images
    newName = sprintf('%01d.jpg', i); % New file name
    imwrite(sf, newName); % Save in the current folder
end
```

**Figure 1.2 (b):** Matlab's Resize Function

To streamline the data collection and preprocessing process, each team member is assigned responsibility for one of the four flower categories. Each member is tasked to collect images for their assigned category and undergoes the preprocessing tasks as stated above. This division of tasks will ensure efficiency and consistency across the dataset.



**Figure 1.2 (c): 100 Different Images of Lilac Dataset**



**Figure 1.2 (d): 100 Different Images of Daisy Dataset**



**Figure 1.2 (e):** 100 Different Images of Tulip Dataset



**Figure 1.2 (f):** 100 Different Images of Lily Dataset

### 1.3 | Output Classes

In the context of this Flower classification project, the output class corresponds to the flower type that the model will predict. To ensure that the model provides correct predictions, the data set is organized into four different output categories or classes. Each output class corresponds to one of the flower types that the model will predict. The output classes are:

- i) Lilac
- ii) Daisy
- iii) Tulip
- iv) Lily

Each of these classes is represented by a folder containing 100 flower images each. These folders are used to organize the input data, and the model is trained to classify images into one of these four classes based on features learned during training. The model's task is to assign the correct label to each flower image it processes. Meaning to say that for each input image, the model will output one of these four classes based on the flower type it predicts.

| Name                                                                                      | Date modified   | Type        |
|-------------------------------------------------------------------------------------------|-----------------|-------------|
|  daisy | 24-Dec-24 10:47 | File folder |
|  lilac | 24-Dec-24 10:45 | File folder |
|  lily  | 24-Dec-24 10:44 | File folder |
|  tulip | 24-Dec-24 10:46 | File folder |

**Figure 1.3 (a):** Output Folders For Each Class



## 1.4 | Segregation of Data

The dataset consists of 404 images, with 101 images for each of the four flower categories. Of these, 100 images for each class are used for training and validation, and one image for testing. Hence, the dataset can be said divided into three subsets as follows:

### i) Training Dataset (75%):

- A total of 300 images (approximately 75% of the dataset) are used for the training phase.
- The model learns the characteristics and patterns of each flower category with the aid of these pictures.

### ii) Validation Dataset (25%):

- A total of 100 images (approximately 25% of the dataset) are used for validation.
- The validation dataset is used to evaluate the model performance during the training process.
- This dataset will be used for the model to make initial predictions helping to avoid overfitting, a situation where the model memorizes the training data too much resulting in low accuracy. This will help to make sure that the model works well on new images during the testing phase.

### iii) Testing Dataset:

- A total of 4 images (1 image per category) are reserved for testing.
- These images are completely excluded from the training and validation processes and are used to provide an unbiased evaluation of the model's performance on unseen data.

## 2.0 TRAINING PHASE

A total of 12 experiments are conducted, each using a unique CNN architecture and parameter configurations. The objective is to identify the architecture and parameter combination that yields the highest accuracy.

### 2.1 | Before Training

- Loading the Dataset

Before the training begins, the dataset will be loaded first. This step is significant to ensure that Matlab correctly identifies and loads the images from the specified dataset path.

```
% Load the dataset

flowerDatasetPath = fullfile('D:\Flower
Classification\Flowers');

imds = imageDatastore(flowerDatasetPath, 'IncludeSubfolders',
true, 'LabelSource', 'foldernames');
```

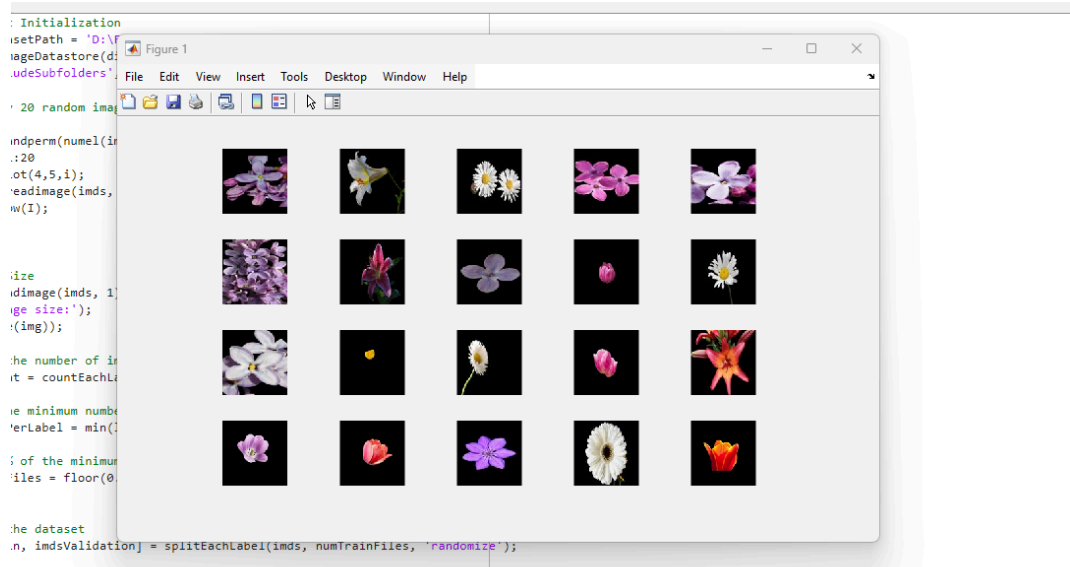
**Figure 2.1 (a):** Code for Loading the Dataset

- Displaying 20 Random Images

To verify the dataset has been loaded correctly, 20 random images will be displayed. By showing images from each class, we can confirm that the data corresponds to the correct flower categories, ensuring there are no mislabeling or incorrect data paths.

```
% Display 20 random images
figure;
perm = randperm(numel(imds.Files), 20);
for i = 1:20
    subplot(4,5,i);
    I = readimage(imds, perm(i));
    imshow(I);
end
```

**Figure 2.1 (b):** Code for Displaying 20 Random Images of Flowers



**Figure 2.1 (c): 20 Random Flowers Displayed**

- Counting Labels

The count of images for each class (label) is checked to determine if the dataset is balanced. A well-balanced dataset ensures that the model has a fair chance to learn from each class equally, without bias toward any particular one.

```

% Count the number of images for each label
labelCount = countEachLabel(imds);

```

**Figure 2.1 (d): Code to Count The Number Of Images For Each Label**

```
labelCount =  
4x2 table  
      Label      Count  
-----  
      daisy      100  
      lilac      100  
      lily       100  
      tulip      100
```

**Figure 2.1 (e):** Output

- Checking for Image Size

The code below specifies the image size for the network's input layer, ensuring all images are resized to 100x100 pixels. This step standardizes the dataset, ensuring it meets the uniform image dimensions required for model training.

```
% Image Size  
img = readimage(imds, 1);  
disp('Image size:');  
disp(size(img));
```

**Figure 2.1 (f):** Code to Check For Image Size

```
ans =  
100  100  3
```

**Figure 2.1 (g):** Output

- Splitting the dataset

The dataset will be split into two parts which are training and validation. Firstly, The code checks the minimum number of images available in any label (flower category). This ensures that the model does not end up training with an imbalanced number of images from each class.

```
% Find the minimum number of images across all labels  
minFilesPerLabel = min(labelCount.Count);
```

**Figure 2.1 (h):** Code to Find The Minimum Number of Image

To ensure the model has enough data for training while still preserving some data for validation, 75% of the images from each class are used for training.

```
% Use 75% of the minimum count for training  
numTrainFiles = floor(0.75 * minFilesPerLabel);
```

**Figure 2.1 (i):** Code to Allocate 75% of Images for Training

The dataset is randomly split into a training set and a validation set, with 75% used for training and the remaining 25% for validation. This allows the model to learn from the training data and then be tested on unseen data during training to check for generalization.

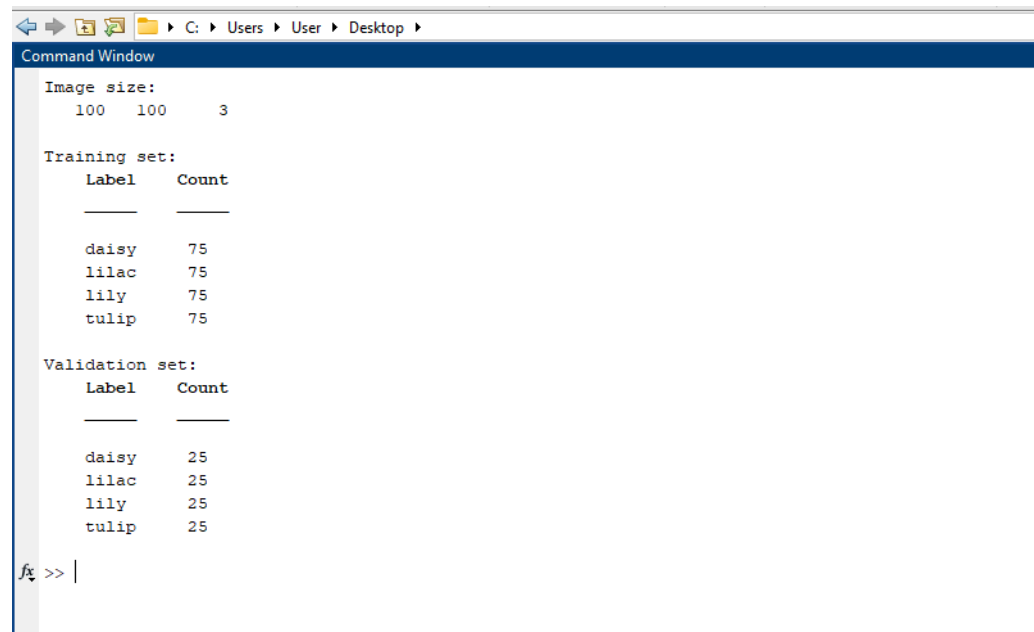
```
% Split the dataset  
[imdsTrain, imdsValidation] = splitEachLabel(imds,  
numTrainFiles, 'randomize');
```

**Figure 2.1 (j):** Code to Split The Dataset

Finally, the number of images in the training and validation sets is displayed to confirm that the dataset has been properly split.

```
% Display the number of files in each set  
disp('Training set:');  
disp(countEachLabel(imdsTrain));  
disp('Validation set:');  
disp(countEachLabel(imdsValidation));
```

**Figure 2.1 (k):** Code For Displaying 20 Random Images Of Flowers



```
Image size:  
100 100 3  
  
Training set:  
Label Count  
-----  
daisy 75  
lilac 75  
lily 75  
tulip 75  
  
Validation set:  
Label Count  
-----  
daisy 25  
lilac 25  
lily 25  
tulip 25  
  
fx >> |
```

**Figure 2.1 (l):** Full Output

## 2.2 | Experiment 1

In training a neural network, experiment 1 involves defining the CNN architecture as the initial network. Below are the details about network architecture for experiment 1.

### Network Architecture:

```
%Experiment 1
layers = [

    imageInputLayer([100 100 3])

    convolution2dLayer(4, 50, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(4, 85, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(4, 100, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.04, ...
    'MaxEpochs', 10, ...
    'MiniBatchSize', 30, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.2 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers: 13

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.04
- Epochs: 10
- Validation Frequency: Every 30 iterations

**Figure 2.2 (b):** Summary of Network Architecture and Training Configuration

### Explanation:

```
convolution2dLayer(4, 50, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.2 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 50 filters of size [4x4] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(4, 85, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.2 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 85 filters of size [4x4] and 'same' padding, ensuring that the output is the same size as the input. This is



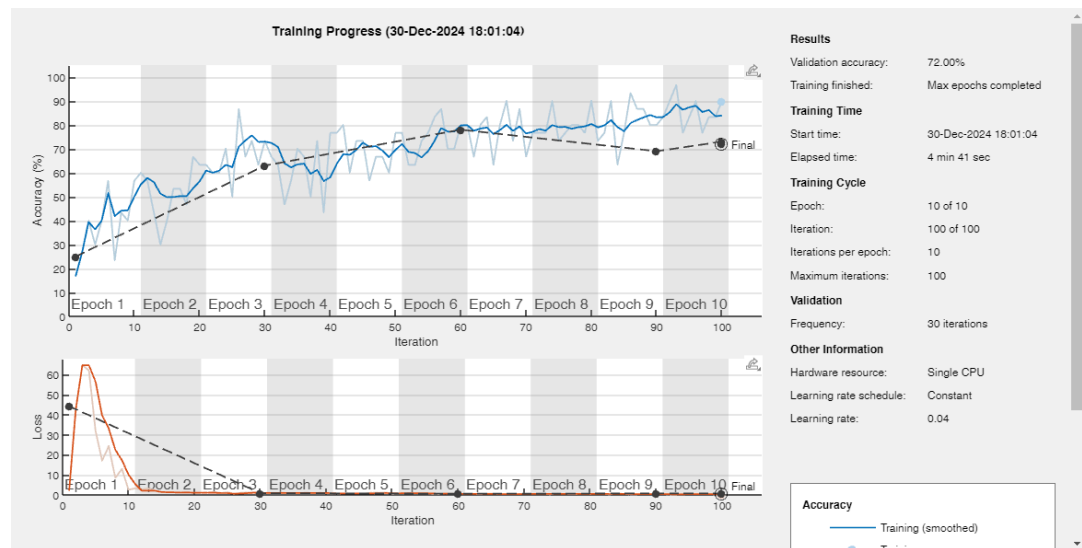
followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(4, 100, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.2 (e): Convolutional Block 3**

This block of codes creates a 2-D convolutional layer with 100 filters of size [4x4] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

### Training Progress:



**Figure 2.2 (f): Training Progress**

Figure 2.2 (d) below illustrates the training progress, showing the network's performance over 4 epochs. For Experiment 1, the validation accuracy achieved is 72.00%.

## 2.3 | Experiment 2

### Network Architecture:

```
%Experiment 2

layers = [

    imageInputLayer([100 100 3])

    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    reluLayer

    convolution2dLayer(3,32,'Padding','same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(1,'Stride',1)

    convolution2dLayer(3,32,'Padding','same')
    batchNormalizationLayer

    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    maxPooling2dLayer(2,'Stride',2)

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer

];

options = trainingOptions('sgdm', ...
    'InitialLearnRate',0.0006, ...
    'MaxEpochs',4, ...
    'Shuffle','every-epoch', ...
    'ValidationData',imdsValidation, ...
    'ValidationFrequency',25, ...
    'Verbose',false, ...
    'Plots','training-progress');

net = trainNetwork(imdsTrain,layers,options);
```

**Figure 2.3 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers:

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.0006
- Epochs: 4
- Validation Frequency: Every 25 iterations

**Figure 2.3 (b):** Summary of Network Architecture and Training Configuration

#### Explanation:

```
convolution2dLayer(3,8,'Padding','same')  
batchNormalizationLayer  
reluLayer
```

**Figure 2.3 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 8 filters of size [3x3] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a batchNormalizationLayer which helps to stabilize the training. Next is ReLU to introduce non-linearity.

```
convolution2dLayer(3,32,'Padding','same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(1,'Stride',1)
```

**Figure 2.3 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 35 filters of size [3x3] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a batchNormalizationLayer which helps to stabilize the training. Next is ReLU activation layer (reluLayer) and a max pooling layer

(maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3,32,'Padding','same')  
batchNormalizationLayer
```

**Figure 2.3 (e):** Convolutional Block 3

This block of codes creates a 2-D convolutional layer with 32 filters of size [3x3] and ‘same’ padding, ensuring that the output is the same size as the input. This is followed by a batchNormalizationLayer which helps to stabilize the training.

```
convolution2dLayer(3,8,'Padding','same')  
batchNormalizationLayer  
maxPooling2dLayer(2,'Stride',2)
```

**Figure 2.3 (f):** Convolutional Block 4

This block of codes creates a 2-D convolutional layer with 8 filters of size [3x3] and ‘same’ padding, ensuring that the output is the same size as the input. This is followed by a batchNormalizationLayer which helps to stabilize the training, and max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
fullyConnectedLayer(4)  
softmaxLayer  
classificationLayer
```

**Figure 2.3 (g):** Fully Connected Layer

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

Training Progress:

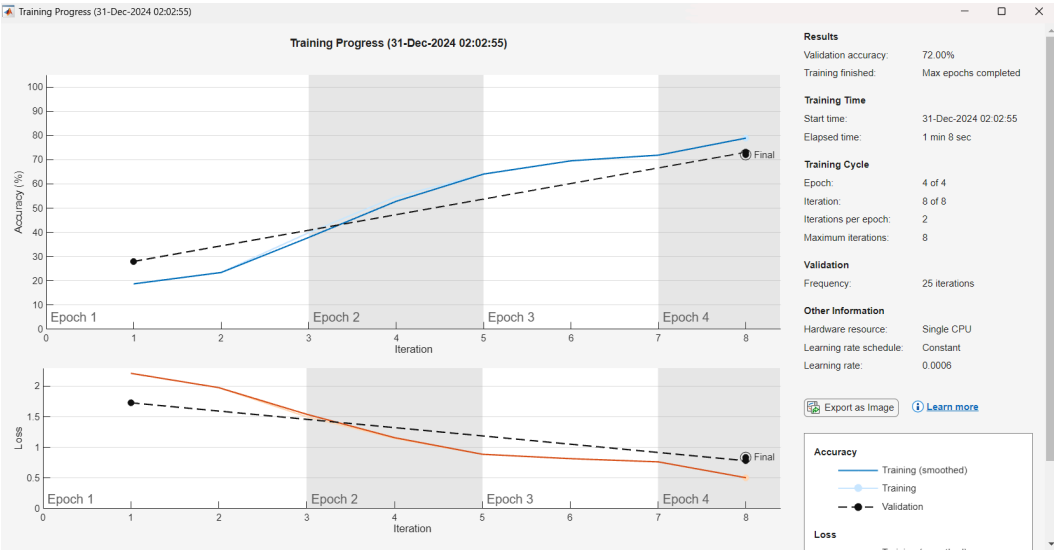


Figure 2.3 (h): Training Progress

Figure 2.3 (h) above illustrates the training progress, showing the network’s performance over 8 epochs. For Experiment 2, the validation accuracy achieved is 72.00%.

## 2.4 | Experiment 3

### Network Architecture:

```
%Experiment 3
layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(3, 25, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 85, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 150, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];
options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.05, ...
    'MaxEpochs', 15, ...
    'MiniBatchSize', 30, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.4 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers:

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.05
- Epochs: 15
- Validation Frequency: Every 30 iterations

**Figure 2.4 (b):** Summary of Network Architecture and Training Configuration

### Explanation:

```
convolution2dLayer(3, 25, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.4 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 25 filters of size [3x3] and ‘same’ padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3, 85, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.4 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 50 filters of size [3x3] and ‘same’ padding, ensuring that the output is the same size as the input. This is

followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3, 150, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.4 (e):** Convolutional Block 3

This block of codes creates a 2-D convolutional layer with 150 filters of size [3x3] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
fullyConnectedLayer(4)
softmaxLayer
classificationLayer
```

**Figure 2.4 (f):** Fully Connected Layer

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.



Training Progress:

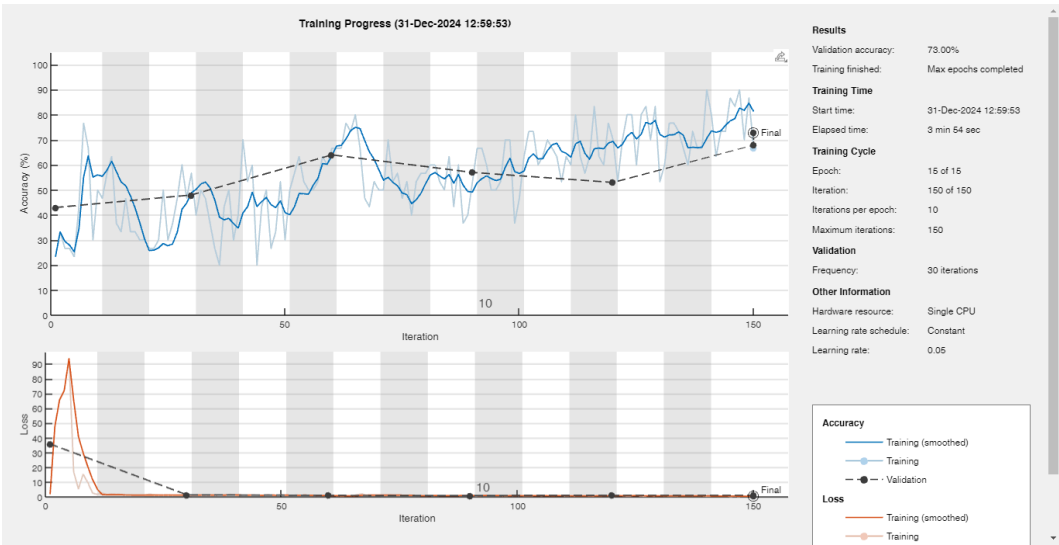


Figure 2.4 (g): Training Progress

Figure 2.4 (g) below illustrates the training progress, showing the network’s performance over 4 epochs. For Experiment 3, the validation accuracy achieved is 73.00%.

## 2.5 | Experiment 4

### Network Architecture:

```
%Experiment 4

layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2,'Stride',2)

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer];

options = trainingOptions('sgdm', ...
    'InitialLearnRate',0.0006, ...
    'MaxEpochs',18, ...
    'Shuffle','every-epoch', ...
    'ValidationData',imdsValidation, ...
    'ValidationFrequency',15, ...
    'Verbose',false, ...
    'Plots','training-progress');

net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.5 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers:

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.0006
- Epochs: 18
- Validation Frequency: Every 15 iterations

**Figure 2.5 (b):** Summary of Network Architecture and Training Configuration

**Explanation:**

```
convolution2dLayer(3,8,'Padding','same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2,'Stride',2)
```

**Figure 2.5 (c):** Convolutional Block 1

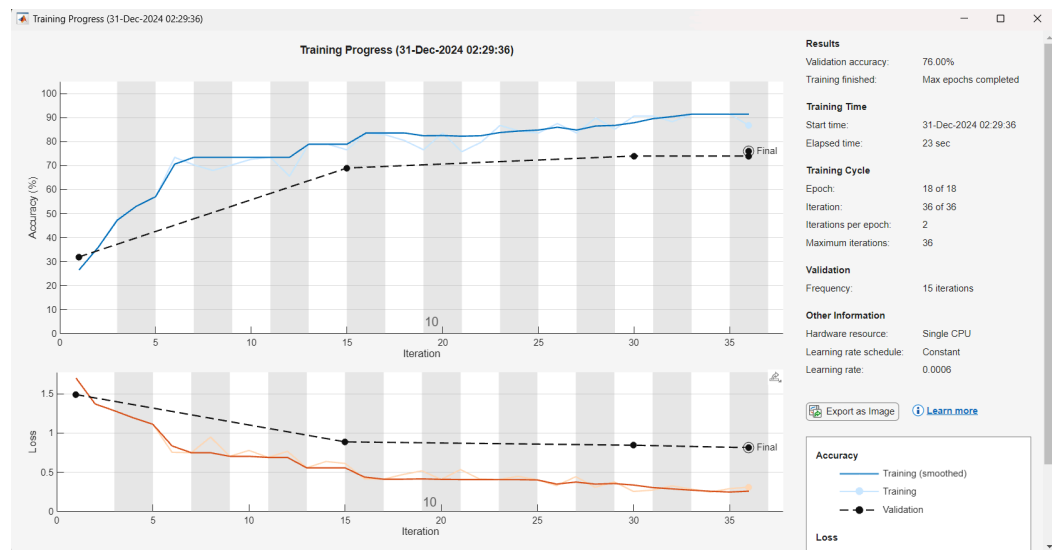
This block of codes creates a 2-D convolutional layer with 8 filters of size [3x3] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a batchNormalizationLayer which helps to stabilize the training. Next is ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
fullyConnectedLayer(4)  
softmaxLayer  
classificationLayer
```

**Figure 2.5 (d):** Fully Connected Layer

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

## Training Progress:



**Figure 2.5 (e): Training Progress**

Figure 2.5 (e) above illustrates the training progress, showing the network's performance over 18 epochs. For experiment 4, the validation accuracy achieved is 76.00%.

## 2.6 | Experiment 5

### Network Architecture:

```
%Experiment 5

layers = [

    imageInputLayer([100 100 3])

    convolution2dLayer(3, 8, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 16, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.001, ...
    'MaxEpochs', 9, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.6 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers: 17

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.001
- Epochs: 9
- Validation Frequency: Every 30 iterations

**Figure 2.6 (b):** Summary of Network Architecture and Training Configuration

### Explanation:

```
convolution2dLayer(3, 8, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.6 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 8 filters of size [3x3] and 'same' padding, ensuring the output size remains the same as the input. This is followed by a batch normalization layer (batchNormalizationLayer) to stabilize training, a ReLU activation layer (reluLayer) to introduce non-linearity, and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2 to reduce the spatial dimensions by half, thus decreasing computational complexity while preserving important features.

```
convolution2dLayer(3, 16, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer
```

**Figure 2.6 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 16 filters of size [3x3] and ‘same’ padding, ensuring the output size remains the same as the input. This is followed by a batch normalization layer (batchNormalizationLayer) to stabilize training and a ReLU activation layer (reluLayer) to introduce non-linearity.

```
convolution2dLayer(3, 32, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.6 (e):** Convolutional Block 3

This block of codes creates a 2-D convolutional layer with 32 filters of size [3x3] and ‘same’ padding, ensuring the output size remains the same as the input. This is followed by a batch normalization layer (batchNormalizationLayer) to stabilize training, a ReLU activation layer (reluLayer) for non-linearity, and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2 to reduce the spatial dimensions.

```
convolution2dLayer(3, 32, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer
```

**Figure 2.6 (f):** Convolutional Block 4

This block of codes creates a 2-D convolutional layer with 32 filters of size [3x3] and ‘same’ padding, ensuring the output size matches the input size. This is followed by a batch normalization layer (batchNormalizationLayer) to stabilize and accelerate training, and a ReLU activation layer (reluLayer) to introduce non-linearity.

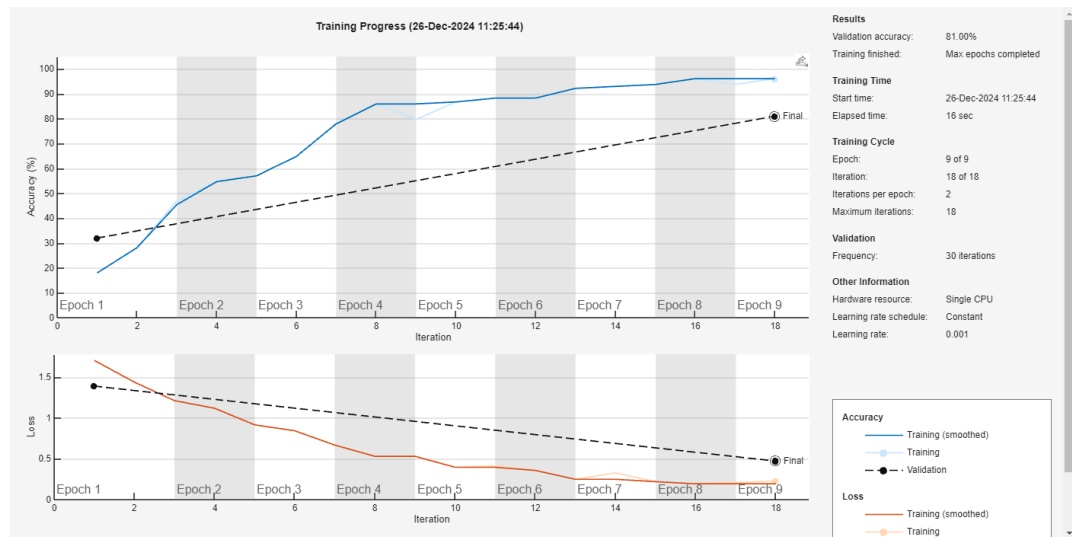
```
fullyConnectedLayer(4)  
softmaxLayer  
classificationLayer
```

**Figure 2.6 (g):** Fully Connected Layer

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.



## Training Progress:



**Figure 2.6 (h): Training Progress**

Figure 2.6 (h) above illustrates the training progress, showing the network's performance over 9 epochs. For Experiment 5, the validation accuracy achieved is 81.00%.

## 2.7 | Experiment 6

### Network Architecture:

```
%Experiment 6

layers = [

    imageInputLayer([100 100 3])

    convolution2dLayer(4,35,'Padding','same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(3,'Stride',3)

    convolution2dLayer(2,30,'Padding','same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer

];

options = trainingOptions('sgdm', ...
    'InitialLearnRate',0.0005, ...
    'MaxEpochs',24, ...
    'Shuffle','every-epoch', ...
    'ValidationData',imdsValidation, ...
    'ValidationFrequency',38, ...
    'Verbose',false, ...
    'Plots','training-progress');

net = trainNetwork(imdsTrain,layers,options);
```

**Figure 2.7 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers:

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.0005
- Epochs: 24
- Validation Frequency: Every 38 iterations

**Figure 2.7 (b):** Summary of Network Architecture and Training Configuration

### Explanation:

```
convolution2dLayer(4,35,'Padding','same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(3,'Stride',3)
```

**Figure 2.7 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 35 filters of size [3x3] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a batchNormalizationLayer which helps to stabilize the training. Next is ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [3x3] and a stride of 3.

```
convolution2dLayer(2,30,'Padding','same')
batchNormalizationLayer
reluLayer
```

**Figure 2.7 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 30 filters of size [3x3] and 'same' padding, ensuring that the output is the same size as the input. This is

followed by a batchNormalizationLayer which helps to stabilize the training, and ReLU activation layer (reluLayer).

```
fullyConnectedLayer(4)
softmaxLayer
classificationLayer
```

Figure 2.7 (e): Fully Connected Layer

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

Training Progress:

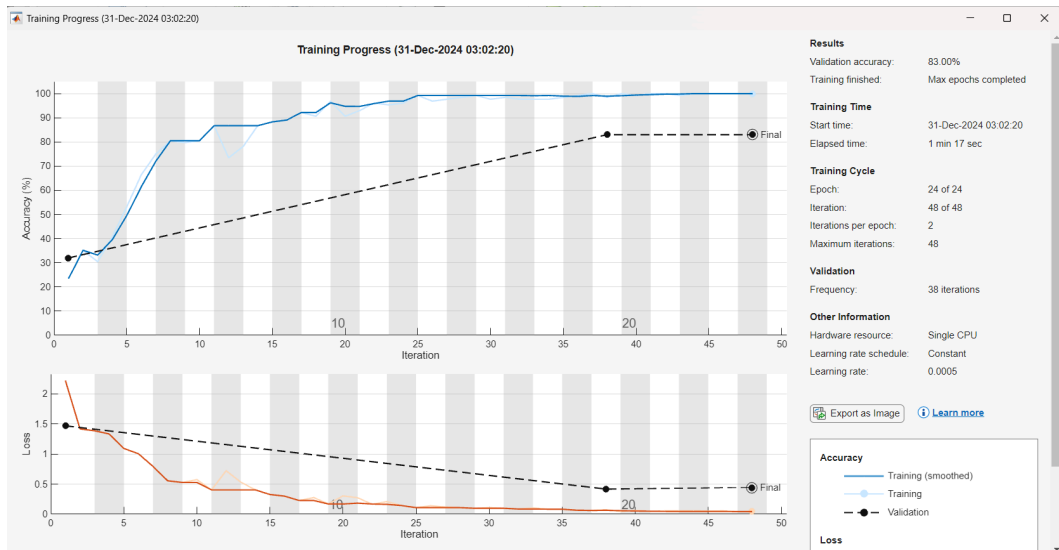


Figure 2.7 (f): Training Progress

Figure 2.7 (f) above illustrates the training progress, showing the network’s performance over 24 epochs. For experiment 6, the validation accuracy achieved is 83.00%.

## 2.8 | Experiment 7

### Network Architecture:

```
%Experiment 7

layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(1, 10, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 38, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(5, 50, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(9, 66, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)
    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.004, ...
    'MaxEpochs', 30, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.8 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers:

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.004
- Epochs: 30
- Validation Frequency: Every 30 iterations

**Figure 2.8 (b):** Summary of Network Architecture and Training Configuration

### Explanation:

```
convolution2dLayer(1, 10, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.8 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 10 filters of size [1x1] and 'same' padding, ensuring the output size remains the same as the input. This is followed by a batch normalization layer (batchNormalizationLayer) to stabilize training, a ReLU activation layer (reluLayer) to introduce non-linearity, and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2 for downsampling.

```
convolution2dLayer(3, 38, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.8 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 38 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(5, 50, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.8 (e):** Convolutional Block 3

This block of codes creates a 2-D convolutional layer with 50 filters of size [5x5] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(9, 66, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.8 (f):** Convolutional Block 4

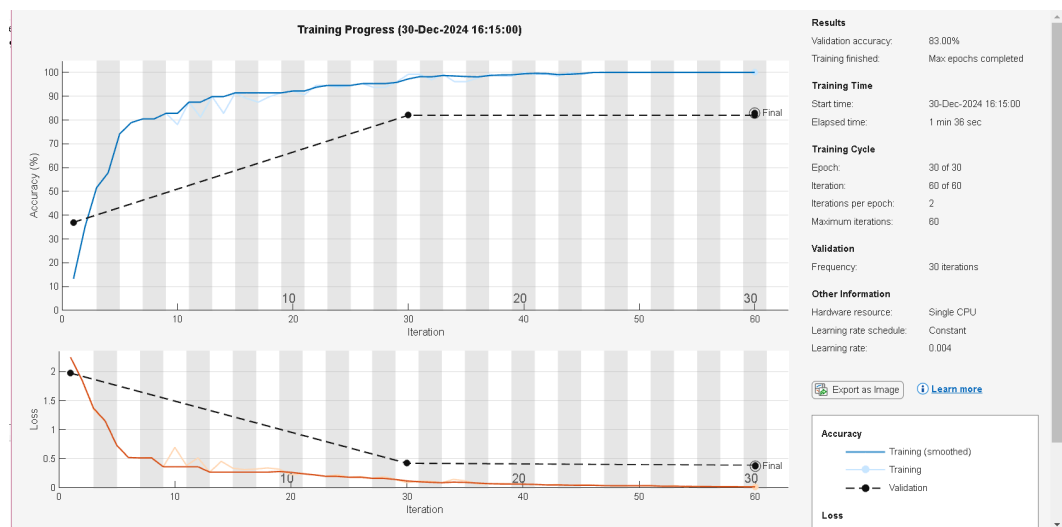
This block of codes creates a 2-D convolutional layer with 66 filters of size [9x9] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
fullyConnectedLayer(4)
softmaxLayer
classificationLayer
```

**Figure 2.8 (g):** Fully Connected Layer

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

### Training Progress:



**Figure 2.8 (h):** Training Progress

Figure 2.8 (h) below illustrates the training progress, showing the network's performance over 4 epochs. For Experiment 7, the validation accuracy achieved is 83.00%.



## 2.9 | Experiment 8

### Network Architecture:

```
%Experiment 8

layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(5, 128, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 16, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(1, 'Stride', 1)

    convolution2dLayer(3, 128, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(1, 'Stride', 1)

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.001, ...
    'MaxEpochs', 9, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.9 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers: 22

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.001
- Epochs: 9
- Validation Frequency: Every 30 iterations

**Figure 2.9 (b):** Summary of Network Architecture and Training Configuration

**Explanation:**

```
convolution2dLayer(5, 128, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.9 (c):** Convolutional Block 1

This block of codes creates a convolutional layer with 128 filters of size [5x5] and 'same' padding, followed by batch normalization, ReLU activation, and max pooling with a 2x2 window and stride 2 to reduce spatial dimensions.

```
convolution2dLayer(3, 16, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer
```

**Figure 2.9 (d):** Convolutional Block 2

This block of codes creates a convolutional layer with 16 filters of size [3x3] and 'same' padding, followed by batch normalization and ReLU activation for non-linearity.

```
convolution2dLayer(3, 32, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(1, 'Stride', 1)
```

**Figure 2.9 (e):** Convolutional Block 3

This block of codes creates a convolutional layer with 32 filters of size [3x3] and ‘same’ padding, followed by batch normalization, ReLU activation, and max pooling with a 1x1 window and stride 1.

```
convolution2dLayer(3, 128, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(1, 'Stride', 1)
```

**Figure 2.9 (f):** Convolutional Block 4

This block of codes creates a convolutional layer with 128 filters of size [3x3] and ‘same’ padding, followed by batch normalization, ReLU activation, and max pooling with a 1x1 window and stride 1.

```
convolution2dLayer(3, 32, 'Padding', 'same')
batchNormalizationLayer
reluLayer
```

**Figure 2.9 (g):** Convolutional Block 5

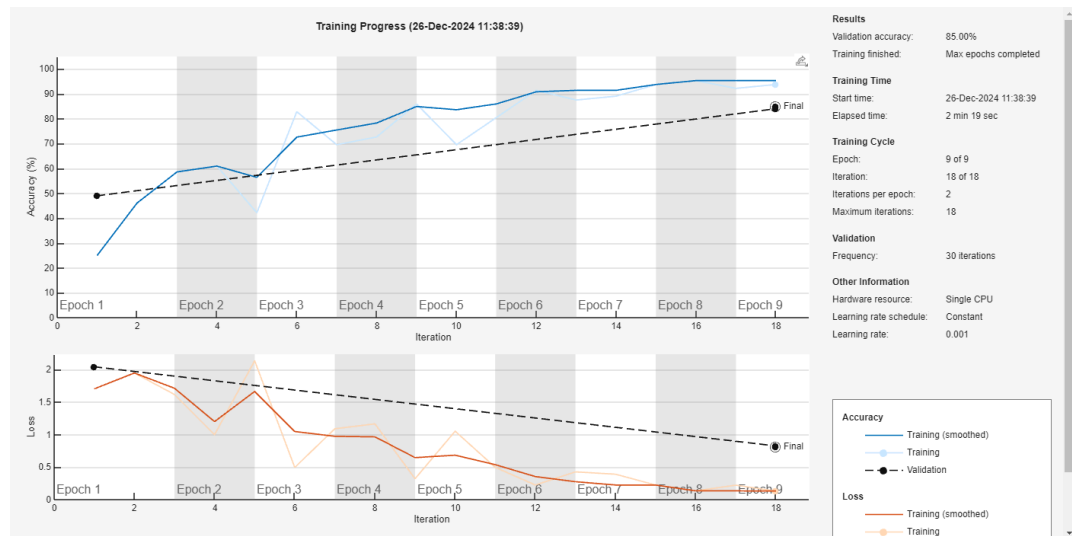
This block of codes creates a convolutional layer with 32 filters of size [3x3] and ‘same’ padding, followed by batch normalization and ReLU activation.

```
fullyConnectedLayer(4)
softmaxLayer
classificationLayer
```

**Figure 2.9 (h): Fully Connected Layer**

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

### Training Progress:



**Figure 2.9 (i): Training Progress**

Figure 2.9 (i) above illustrates the training progress, showing the network's performance over 9 epochs. For Experiment 8, the validation accuracy achieved is 85.00%.

## 2.10 | Experiment 9

### Network Architecture:

```
%Experiment 9

layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 64, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 128, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 256, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.001, ...
    'MaxEpochs', 20, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');
net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.10 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers:

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.001
- Epochs: 20
- Validation Frequency: Every 30 iterations

**Figure 2.10 (b):** Summary of Network Architecture and Training Configuration

### Explanation:

```
convolution2dLayer(3, 32, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.10 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 32 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3, 64, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.10 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 64 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3, 128, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.10 (e):** Convolutional Block 3

This block of codes creates a 2-D convolutional layer with 128 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3, 256, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.10 (f):** Convolutional Block 4

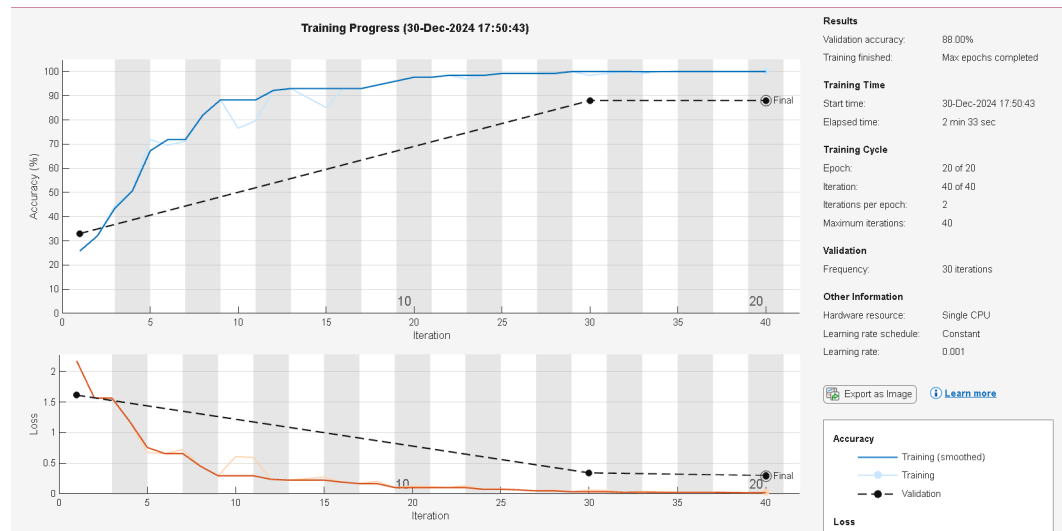
This block of codes creates a 2-D convolutional layer with 256 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
fullyConnectedLayer(4)
softmaxLayer
classificationLayer
```

**Figure 2.10 (g): Fully Connected Layer**

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

### Training Progress:



**Figure 2.10 (h): Training Progress**

Figure 2.10 (h) below illustrates the training progress, showing the network's performance over 20 epochs. For Experiment 9, the validation accuracy achieved is 88.00%.



## 2.11 | Experiment 10

### Network Architecture:

```
%Experiment 10

layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(3, 55, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 65, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 145, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 235, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)
    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.0003, ...
    'MaxEpochs', 15, ...
    'MiniBatchSize', 30, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.11 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers: 17

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.0003
- Epochs: 30
- Validation Frequency: Every 30 iterations

**Figure 2.10 (b):** Summary of Network Architecture and Training Configuration

### Explanation:

```
convolution2dLayer(3, 55, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.11 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 55 filters of size [4x4] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3, 65, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.11 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 65 filters of size [4x4] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3, 145, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.11 (e):** Convolutional Block 3

This block of codes creates a 2-D convolutional layer with 145 filters of size [3x3] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
convolution2dLayer(3, 235, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.11 (f):** Convolutional Block 4

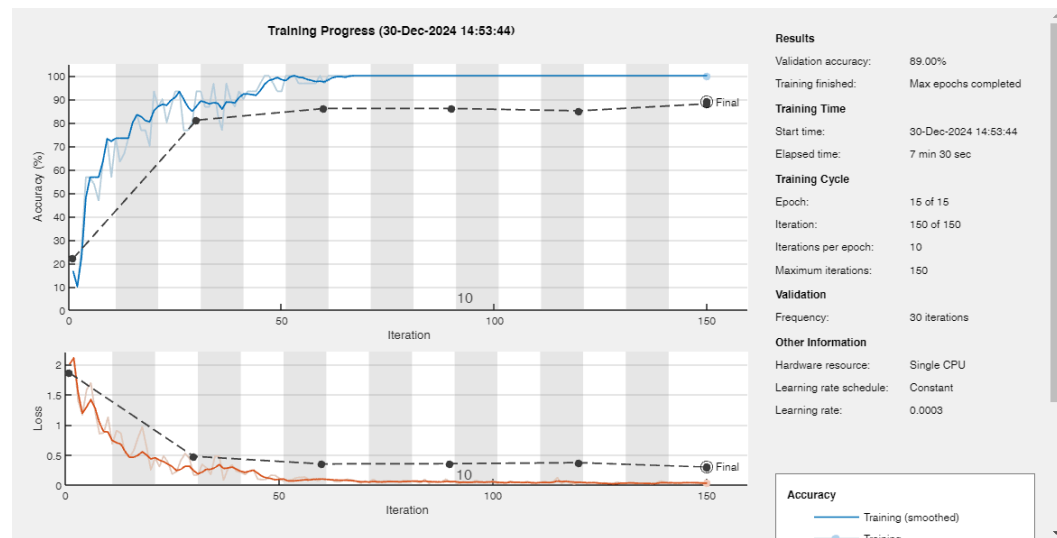
This block of codes creates a 2-D convolutional layer with 235 filters of size [3x3] and 'same' padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [2x2] and a stride of 2.

```
fullyConnectedLayer(4)
softmaxLayer
classificationLayer
```

**Figure 2.11 (g): Fully Connected Layer**

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

### Training Progress:



**Figure 2.11 (h): Training Progress**

Figure 2.11 (h) below illustrates the training progress, showing the network's performance over 4 epochs. For Experiment 10, the validation accuracy achieved is 89.00%.

## 2.12 | Experiment 11

### Network Architecture:

```
%Experiment 11

layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(3, 16, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(3, 'Stride', 2)

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(3, 'Stride', 2)

    convolution2dLayer(3, 64, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(3, 'Stride', 2)

    convolution2dLayer(3, 128, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(3, 'Stride', 2)
    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer

];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.001, ...
    'MaxEpochs', 20, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.12 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers:

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.001
- Epochs: 20
- Validation Frequency: Every 30 iterations

**Figure 2.12 (b):** Summary of Network Architecture and Training Configuration

### Explanation:

```
convolution2dLayer(3, 16, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(3, 'Stride', 2)
```

**Figure 2.12 (c):** Convolutional Block 1

This block of codes creates a 2-D convolutional layer with 16 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [3x2] and a stride of 2.

```
convolution2dLayer(3, 32, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(3, 'Stride', 2)
```

**Figure 2.12 (d):** Convolutional Block 2

This block of codes creates a 2-D convolutional layer with 32 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [3x2] and a stride of 2.

```
convolution2dLayer(3, 64, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(3, 'Stride', 2)
```

**Figure 2.12 (e): Convolutional Block 3**

This block of codes creates a 2-D convolutional layer with 64 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [3x2] and a stride of 2.

```
convolution2dLayer(3, 128, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(3, 'Stride', 2)
```

**Figure 2.12 (f): Convolutional Block 4**

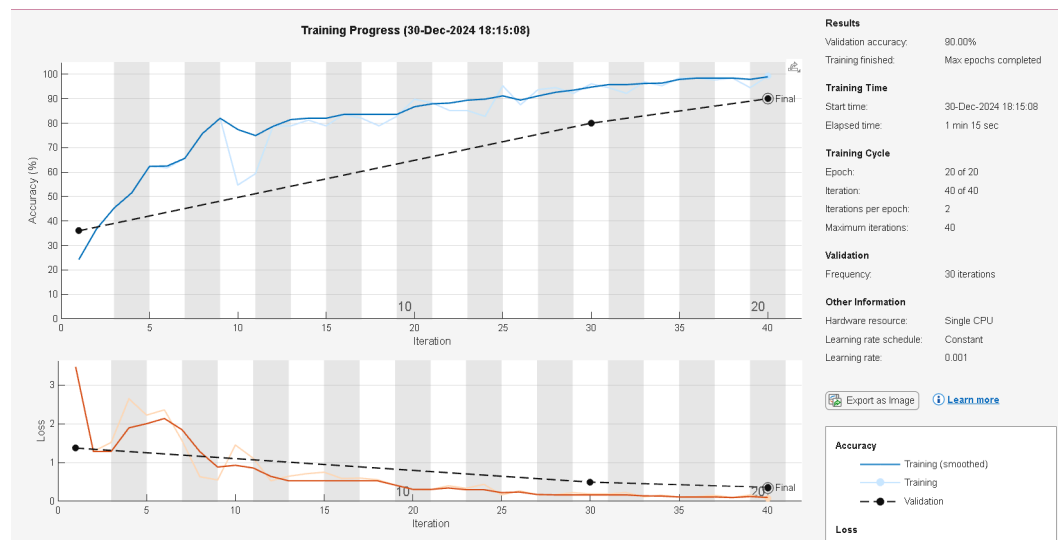
This block of codes creates a 2-D convolutional layer with 128 filters of size [3x3] and same padding, ensuring that the output is the same size as the input. This is followed by a ReLU activation layer (reluLayer) and a max pooling layer (maxPooling2dLayer) with a pool size of [3x2] and a stride of 2.

```
fullyConnectedLayer(4)
softmaxLayer
classificationLayer
```

**Figure 2.12 (g): Fully Connected Layer**

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

### Training Progress:



**Figure 2.13 (h): Training Progress**

Figure 2.12 (h) below illustrates the training progress, showing the network's performance over 20 epochs. For Experiment 11, the validation accuracy achieved is 90.00%.



## 2.13 | Experiment 12

### Network Architecture:

```
%Experiment 12

layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 64, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 128, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 256, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.0005, ...
    'MaxEpochs', 20, ...
    'MiniBatchSize', 32, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

net = trainNetwork(imdsTrain, layers, options);
```

**Figure 2.13 (a):** Network Architecture

Input Image:  $100 \times 100 \times 3$  (RGB)

Total Layers: 20

Fully Connected Layer: 4 outputs (4 classes)

Training:

- Optimizer: SGDM
- Learning Rate: 0.0005
- Epochs: 20
- Validation Frequency: Every 30 iterations

**Figure 2.13 (b):** Summary of Network Architecture and Training Configuration

#### Explanation:

```
convolution2dLayer(3, 32, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.13 (c):** Convolutional Block 1

This block of codes creates a convolutional layer with 32 filters of size [3x3] and ‘same’ padding, followed by batch normalization, ReLU activation, and max pooling with a 2x2 window and stride 2 to reduce spatial dimensions.

```
convolution2dLayer(3, 64, 'Padding', 'same')  
batchNormalizationLayer  
reluLayer  
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.13 (d):** Convolutional Block 2

This block of codes creates a convolutional layer with 64 filters of size [3x3] and ‘same’ padding, followed by batch normalization, ReLU activation, and max pooling with a 2x2 window and stride 2 to reduce spatial dimensions.

```
convolution2dLayer(3, 128, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.13 (e):** Convolutional Block 3

This block of codes creates a convolutional layer with 128 filters of size [3x3] and 'same' padding, followed by batch normalization, ReLU activation, and max pooling with a 2x2 window and stride 2.

```
convolution2dLayer(3, 256, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)
```

**Figure 2.13 (f):** Convolutional Block 4

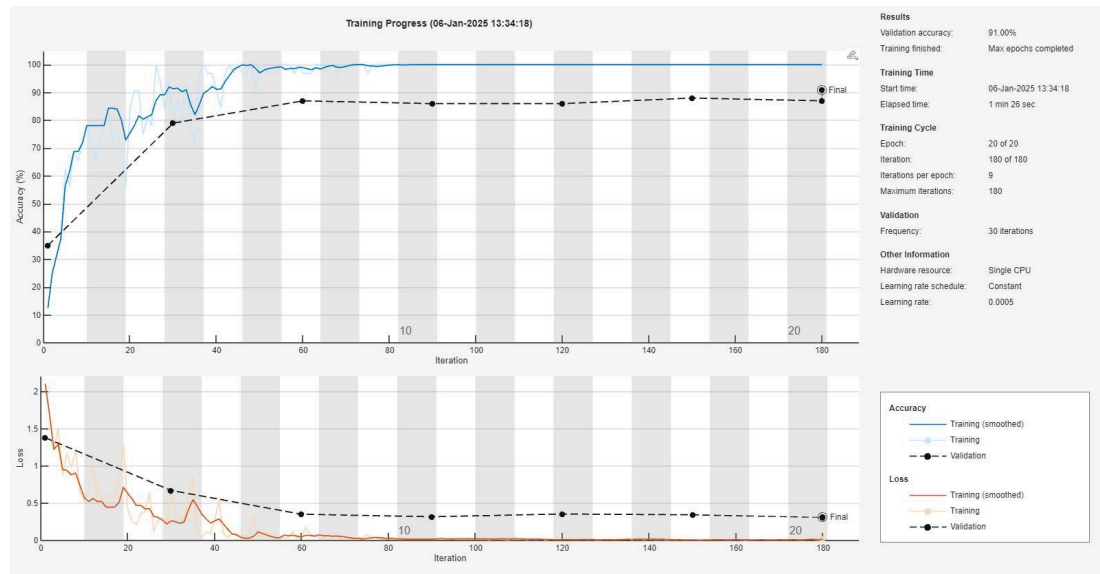
This block of codes creates a convolutional layer with 256 filters of size [3x3] and 'same' padding, followed by batch normalization, ReLU activation, and max pooling with a 2x2 window and stride 2.

```
fullyConnectedLayer(4)
softmaxLayer
classificationLayer
```

**Figure 2.13 (g):** Fully Connected Layer

The fully connected layer connects all previous layers to 4 output units, representing 4 classes. The softmax layer converts the outputs into probabilities, and the classification layer calculates the loss for training.

## Training Progress:



**Figure 2.13 (h): Training Progress**

Figure 2.13 (h) above illustrates the training progress, showing the network's performance over 20 epochs. For Experiment 3, the validation accuracy achieved is 93.00%.

## 3.0 TESTING PHASE

### 3.1 | Final Network Architecture

Of the 12 experiments conducted, the last network from experiment 12 is the most stable and produces the highest accuracy among all network architectures, which is 91%. Hence, this network will be used for the testing phase on four types of flowers.

This network starts with an input layer that processes color images with a resolution of 100 pixels by 100 pixels. It is made up of four blocks, each with a convolutional layer that uses an increasing number of filters, starting with 32, 64, 128, and lastly to 256 to learn progressively more detailed features. Each block includes 3x3 filters with padding to maintain the image size, followed by batch normalization to improve training, a ReLU activation layer for non-linearity, and a max-pooling layer to reduce the image size while keeping important features.

After extracting features, the network has a fully connected layer that outputs predictions for four flower categories. A softmax layer converts these predictions into probabilities, and a classification layer determines the final result. The network is trained using the SGDM optimizer with a learning rate of 0.0005 over 20 epochs. It processes 32 images at a time (mini-batch) and shuffles the data at each epoch to avoid overfitting. Validation data is checked every 30 iterations to track progress, and a live plot shows training metrics.

## Network Architecture:

```
%Experiment 12

layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 64, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 128, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    convolution2dLayer(3, 256, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    fullyConnectedLayer(4)
    softmaxLayer
    classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.0005, ...
    'MaxEpochs', 20, ...
    'MiniBatchSize', 32, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

net = trainNetwork(imdsTrain, layers, options);
```

**Figure 3.1 (a):** Final Network Architecture

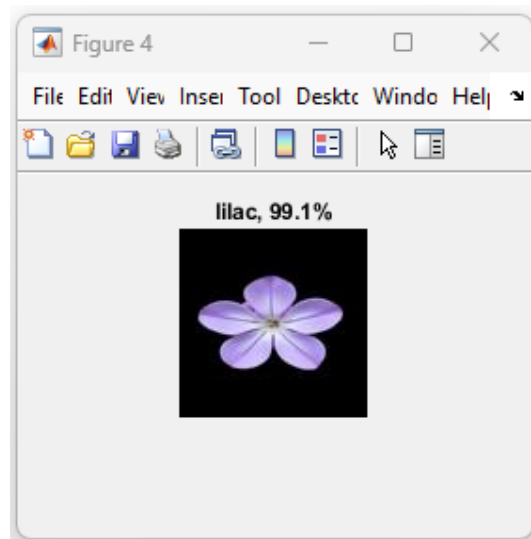
## 3.2 | Testing Steps

Step-by-step to test the trained CNN by using the testing segment code:

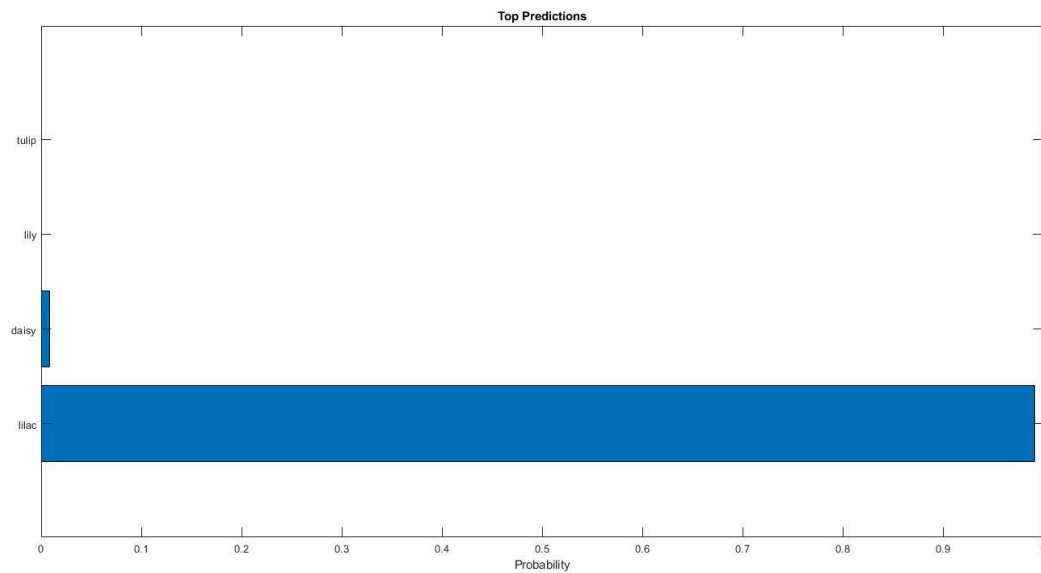
1. Load the Test Dataset  
Define the file path to access the test images, ensuring they are correctly labeled and organized.
2. Image Preprocessing  
Resize the test image to match the network's input size and convert it to RGB if it's in grayscale.
3. Classify Image and Obtain Predictions  
Use the trained CNN model to classify the test image. Obtain the predicted labels and confidence scores.
4. Visualize Predictions and Scores  
Display the test image with the predicted label and corresponding confidence score. Additionally, sort and visualize the top 4 predictions in a bar graph.
5. Repeat Testing  
Repeat the steps for different flower images from the test dataset.

### 3.3 | Testing Results for Lilac

Test image of a Lilac flower is classified by the network. The top prediction, along with its score, the true label, and the predicted label, is shown below.



**Figure 3.3 (a):** Top 4 Prediction of Lilac

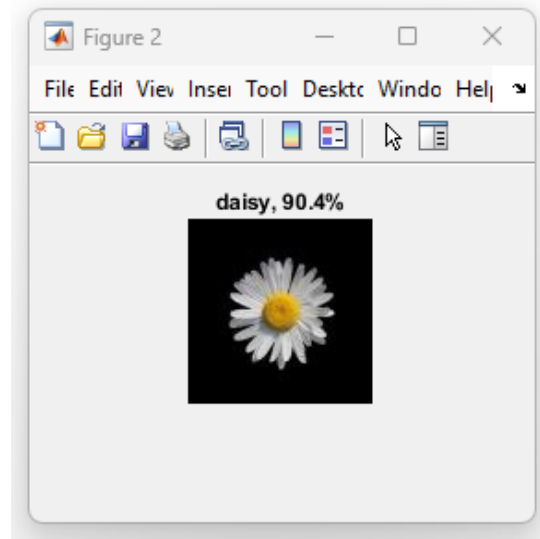


**Figure 3.3 (b):** Probability Percentage of Lilac

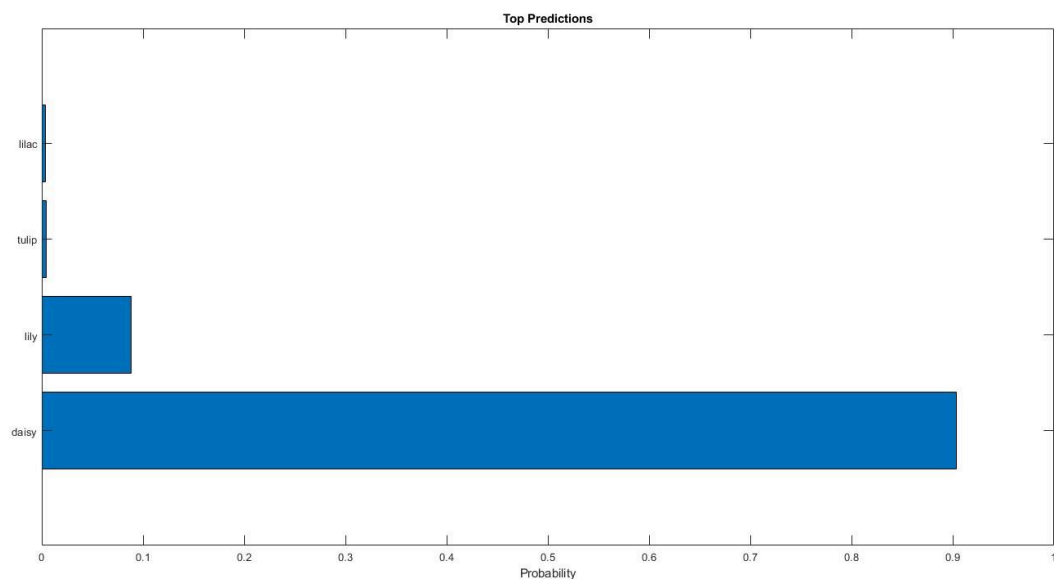


### 3.4 | Testing Results for Daisy

Test image of a Daisy flower is classified by the network. The top prediction, along with its score, the true label, and the predicted label, is shown below.



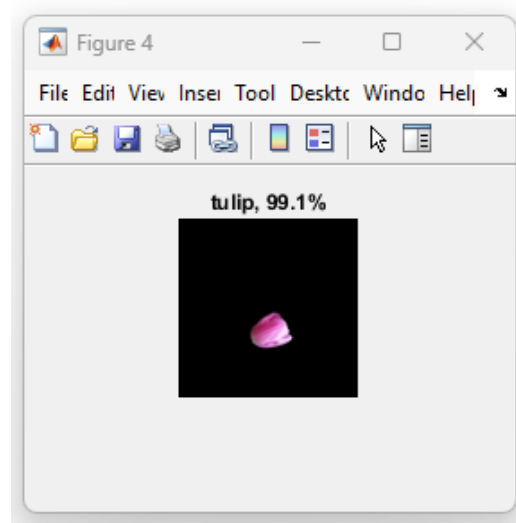
**Figure 3.4 (a):** Top 4 Prediction of Daisy



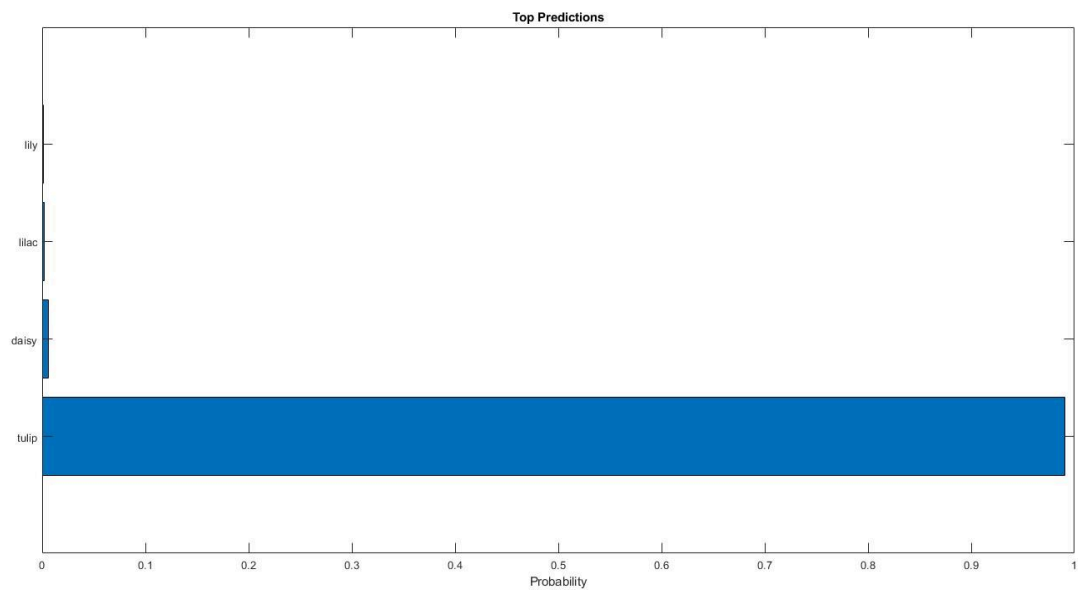
**Figure 3.4 (b):** Probability Percentage of Daisy

### 3.5 | Testing Results for Tulip

Test image of a Tulip flower is classified by the network. The top prediction, along with its score, the true label, and the predicted label, is shown below.



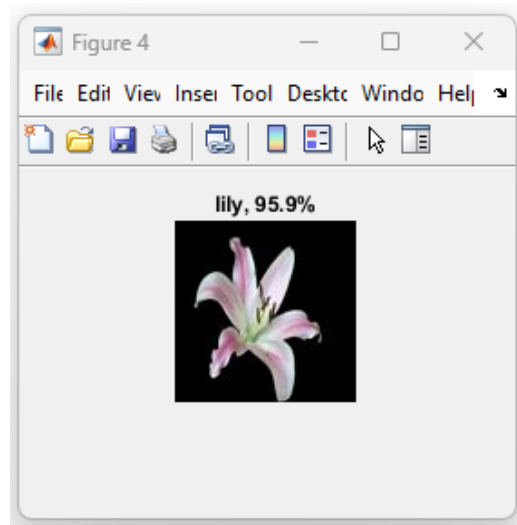
**Figure 3.5 (a):** Top 4 Prediction of Tulip



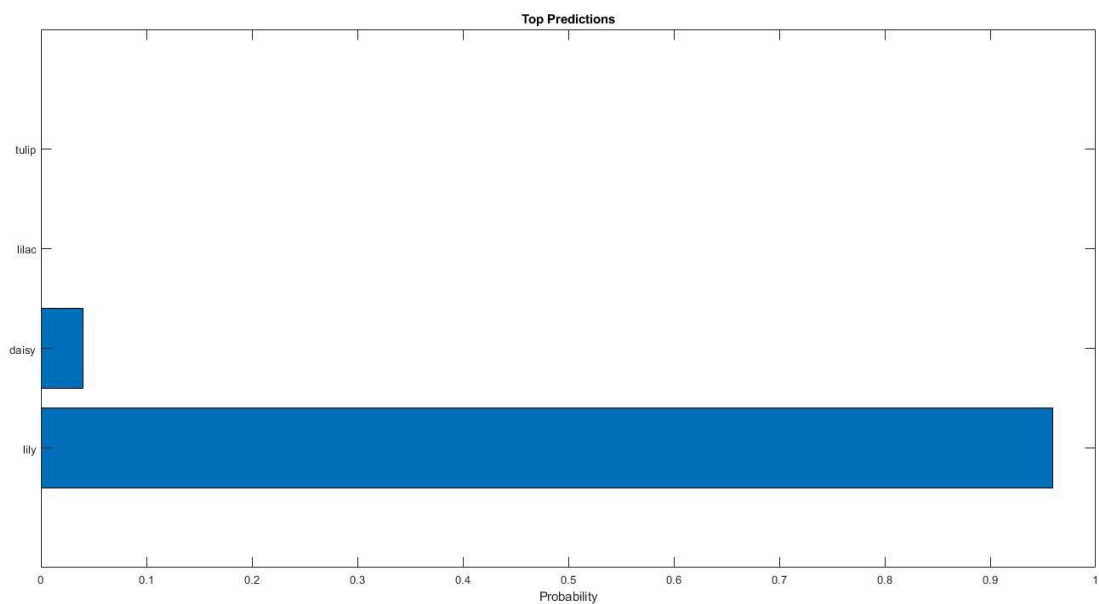
**Figure 3.5 (b):** Probability percentage of Tulip

### 3.6 | Testing Results for Lily

Test image of a Lily flower is classified by the network. The top prediction, along with its score, the true label, and the predicted label, is shown below.



**Figure 3.6 (a):** Top 4 Prediction of Lily



**Figure 3.6 (b):** Probability percentage of Lily

## **4.0 CONCLUSION**

Based on the training and testing results presented, it is evident that the implemented Convolutional Neural Network (CNN) architecture successfully classifies images of lilac, daisy, tulip and lily with a commendable degree of accuracy. The validation accuracy improved progressively through the experiments, with notable performance in recognizing the top predictions for each flower category. The highest accuracy achieved was 91.00% indicating a robust model capable of differentiating between the various flower classes effectively.

In conclusion, the CNN model demonstrated its capability in image classification tasks by accurately predicting and ranking the test images. The consistent performance across different flower categories underscores the model's reliability. Future work could involve fine-tuning the network parameters and expanding the dataset to enhance the model's accuracy and generalizability. Overall, the results validate the effectiveness of the chosen CNN architecture for the classification task, setting a solid foundation for further advancements in this domain.

## 5.0 APPENDIX: MATLAB CODE FOR THE TESTING PHASE

```
%% Test Image Preprocessing and Prediction
inputSize = net.Layers(1).InputSize;

I = imread('D:\Flower Classification\Tester\lily1.jpg'); % Test image path
figure
imshow(I)

% Resize the image if necessary
I = imresize(I,inputSize(1:2));

% Convert to RGB if necessary
if size(I, 3) == 1
    I = repmat(I, [1 1 3]);
End

%% Classification

% Classify the image and get prediction scores
[label, scores] = classify(net, I);
Label;

% Display the image with prediction
figure;
imshow(I);
classNames = net.Layers(end).ClassNames;
title(string(label) + ", " + num2str(100 * scores(classNames == label), 3) +
"%");

% Sort scores in descending order
[~, idx] = sort(scores, 'descend');

% Ensure we only select as many predictions as available (max 4 classes)
numTopPredictions = min(4, numel(scores));
classNamesTop = classNames(idx(1:numTopPredictions));
scoresTop = scores(idx(1:numTopPredictions));

% Display top predictions
figure;
barh(scoresTop);
xlim([0 1]);
title('Top Predictions');
xlabel('Probability');
yticklabels(classNamesTop);
```

## 6.0 APPENDIX: OVERALL MATLAB CODE FOR THE CLASSIFICATION

```
% Dataset Initialization
digitDatasetPath = 'D:\Flower Classification\Flowers'; % Dataset path
imds = imageDatastore(digitDatasetPath, ...
    'IncludeSubfolders', true, 'LabelSource', 'foldernames');

% Display 20 random images
figure;
perm = randperm(numel(imds.Files), 20);
for i = 1:20
    subplot(4, 5, i);
    I = readimage(imds, perm(i));
    imshow(I);
end

% Image Size
img = readimage(imds, 1);
disp('Image size:');
disp(size(img));

% Count the number of images for each label
labelCount = countEachLabel(imds);

% Find the minimum number of images across all labels
minFilesPerLabel = min(labelCount.Count);

% Use 75% of the minimum count for training
numTrainFiles = floor(0.75 * minFilesPerLabel);

% Split the dataset
[imdsTrain, imdsValidation] = splitEachLabel(imds, numTrainFiles,
    'randomize');

% Display the number of files in each set
disp('Training set:');
disp(countEachLabel(imdsTrain));
disp('Validation set:');
disp(countEachLabel(imdsValidation));

%% Training The Network Architecture
layers = [
    imageInputLayer([100 100 3])

    convolution2dLayer(3, 32, 'Padding', 'same')
    batchNormalizationLayer
    reluLayer
```

```

maxPooling2dLayer(2, 'Stride', 2)

convolution2dLayer(3, 64, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)

convolution2dLayer(3, 128, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)

convolution2dLayer(3, 256, 'Padding', 'same')
batchNormalizationLayer
reluLayer
maxPooling2dLayer(2, 'Stride', 2)

fullyConnectedLayer(4)
softmaxLayer
classificationLayer
];

options = trainingOptions('sgdm', ...
    'InitialLearnRate', 0.0005, ...
    'MaxEpochs', 20, ...
    'MiniBatchSize', 32, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', imdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', false, ...
    'Plots', 'training-progress');

net = trainNetwork(imdsTrain, layers, options);

%% Test Image Preprocessing and Prediction
inputSize = net.Layers(1).InputSize;

I = imread('D:\Flower Classification\Tester\lilac1.jpg'); % Test image path
figure
imshow(I)

I = imresize(I, inputSize(1:2)); % Resize the image

% Convert to RGB if necessary
if size(I, 3) == 1
    I = repmat(I, [1 1 3]);
End

```

```

%% Classification

%% Classify the image and get prediction scores
[label, scores] = classify(net, I);
Label;

% Display the image with prediction
figure;
imshow(I);
classNames = net.Layers(end).ClassNames;
title(string(label) + ", " + num2str(100 * scores(classNames == label), 3) +
"%");

% Sort scores in descending order
[~, idx] = sort(scores, 'descend');

% This will select at most 4 predictions
numTopPredictions = min(4, numel(scores));
classNamesTop = classNames(idx(1:numTopPredictions));
scoresTop = scores(idx(1:numTopPredictions));

% Display top predictions
figure;
barh(scoresTop);
xlim([0 1]);
title('Top Predictions');
xlabel('Probability');
yticklabels(classNamesTop);

```